

GUIDE DES FONDAMENTAUX · NIVEAU ZÉRO

Les bases avant OpenShift

Conteneurs, YAML, Kubernetes et OpenShift expliqués depuis le début – avec des schémas, des manifestes commentés et 13 exercices progressifs sur ton cluster.

À lire avant le guide « OpenShift niveau 1 (DO180 / EX180) »

Version 30 septembre 2026

podman

YAML

kubect^l

oc

Document d'étude personnel, non officiel et non affilié à Red Hat, Inc. « Red Hat », « OpenShift » et « Kubernetes » sont des marques de leurs propriétaires respectifs.

SOMMAIRE

Contenu du guide

01	La vue d'ensemble : du programme au cluster	3
02	Conteneurs et images	5
03	Le YAML sans douleur	7
04	Kubernetes : architecture et fonctionnement	10
05	Pods, Deployments et labels	12
06	Réseau : Services et Routes	15
07	Configuration : ConfigMaps et Secrets	17
08	Stockage persistant	19
09	Fiabilité et mises à jour	21
10	Ce qu'OpenShift ajoute	24
11	Premiers pas avec oc et vim	26
12	Rappels réseau utiles	28
13	Exercices niveau zéro (13)	30
14	Glossaire	33
A	Annexe : solutions des exercices	35

COMMENT LIRE CE GUIDE

Chaque chapitre s'appuie sur le précédent : lis-les **dans l'ordre**. À la fin de chaque chapitre, un encadré « Tu dois savoir répondre » te dit si tu peux passer au suivant. Les exercices du chapitre 13 suivent le même ordre ; leurs solutions sont en **annexe**, pour ne pas les avoir sous les yeux.

KUBERNETES OU OPENSIFT ?

OpenShift **est** du Kubernetes, avec des ajouts. Tout ce qui est expliqué aux chapitres 4 à 9 fonctionne sur n'importe quel Kubernetes (K3s, EKS, AKS, GKE). Le chapitre 10 présente ce qui est propre à OpenShift. Dans les exemples, `oc` peut toujours être remplacé par `kubectl`, sauf pour les commandes marquées « OpenShift ».

CHAPITRE 01

La vue d'ensemble : du programme au cluster

Pourquoi les conteneurs, puis pourquoi un orchestrateur.

■ Le problème de départ

Une application a besoin de son code, mais aussi d'une version précise d'un langage, de bibliothèques, de fichiers de configuration. Installée directement sur un serveur, elle entre en conflit avec les autres applications et fonctionne « chez le développeur » mais pas en production.

■ Étape 1 : le conteneur

Un **conteneur** emballe l'application avec tout ce dont elle a besoin, et l'exécute isolée des autres, en partageant le noyau Linux de la machine. Il démarre en une seconde et se comporte pareil partout.

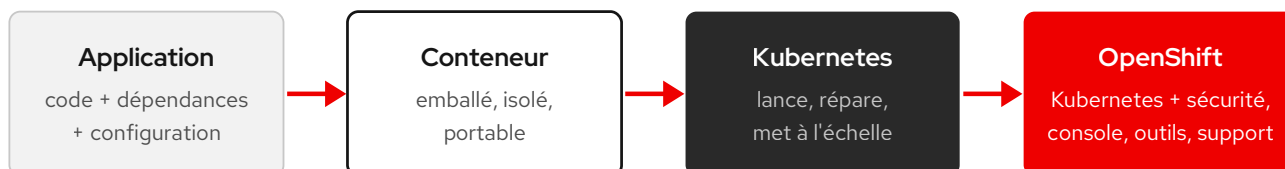
Analogie : le conteneur maritime. Peu importe ce qu'il contient, le port sait le charger, le déplacer et le poser.

■ Étape 2 : l'orchestrateur

Avec 3 conteneurs, on les gère à la main. Avec 300 répartis sur 20 serveurs, il faut un logiciel qui décide où les lancer, les redémarre s'ils tombent, les multiplie quand la charge monte, leur donne une adresse réseau stable et les met à jour sans coupure. C'est le rôle de **Kubernetes**.

■ Étape 3 : la plateforme

Kubernetes seul est une boîte à outils. **OpenShift** est une plateforme complète construite dessus : console web, sécurité renforcée par défaut, authentification, registre d'images, monitoring, mises à jour du cluster, support éditeur.



Chaque étape ajoute une couche au-dessus de la précédente.

CONTENEUR OU MACHINE VIRTUELLE ?

Une VM (comme dans Proxmox) embarque un système d'exploitation complet : lourde, démarrage en minutes. Un conteneur partage le noyau de l'hôte : léger, démarrage en secondes. Ton cluster OpenShift est d'ailleurs une **VM** qui fait tourner des **conteneurs**.

TU DOIS SAVOIR RÉPONDRE

- Quel problème résout un conteneur ? Et un orchestrateur ?
- Quelle est la relation entre Kubernetes et OpenShift ?

Conteneurs et images

Les briques de base, manipulées avec Podman.

■ Image et conteneur

Notion	Définition	Analogie
Image	Modèle en lecture seule : système de fichiers + métadonnées (commande de démarrage, ports, utilisateur). Construite en couches empilées.	Le plan / la recette
Conteneur	Une image en cours d'exécution, avec une couche modifiable par-dessus. On peut lancer plusieurs conteneurs depuis la même image.	Le plat cuisiné à partir de la recette
Registre	Serveur qui stocke et distribue les images (quay.io, registry.redhat.io, Docker Hub, le registre interne d'OpenShift).	La bibliothèque de recettes

■ Lire le nom d'une image

```
registry.access.redhat.com / ubi9 / httpd-24 : latest  
└─── registre ───┘ └─ dépôt (namespace/nom) ┘ └─ tag ┘  
  
registry.access.redhat.com/ubi9/httpd-24@sha256:4f2a... # par digest
```

- **Tag** (`latest` , `1.2`) : étiquette **mobile**, qui peut pointer demain vers une autre image.
- **Digest** (`sha256:...`) : empreinte du contenu, **immuable**. Garantit exactement la même image.
- Sans registre précisé, l'outil cherche dans ses registres par défaut. Donne toujours le nom complet.

■ Ce qu'une image définit

Élément	Rôle
Commande de démarrage	Le processus lancé (<code>ENTRYPOINT</code> / <code>CMD</code>). Quand il se termine, le conteneur s'arrête.
Ports exposés	Les ports sur lesquels l'application écoute (ex. 8080). Purement indicatif, n'ouvre rien.
Utilisateur	Sous quel utilisateur tourne le processus. Crucial sur OpenShift : root est refusé par défaut.
Variables d'environnement	Valeurs par défaut, modifiables au lancement.
Répertoire de travail	Dossier courant du processus.

■ Construire une image : le Containerfile

```
# Containerfile (même syntaxe qu'un Dockerfile)
FROM registry.access.redhat.com/ubi9/httpd-24      # image de base
COPY index.html /var/www/html/                    # ajoute mon fichier
# cette image de base écoute déjà sur 8080 et ne tourne pas en root
```

```
$ podman build -t monsite:1.0 .
```

■ Podman : l'essentiel

Podman s'utilise comme Docker (mêmes commandes), sans démon et sans root. Sur Fedora : `sudo dnf install podman`.

```
podman pull <image>                                # télécharger
podman images                                       # lister les images locales
podman run -d --name web -p 8080:8080 <image>      # lancer en fond, port hôte:conteneur
podman ps [-a]                                     # conteneurs en cours (-a : tous)
podman logs [-f] web                               # journaux
podman exec -it web bash                           # shell dans le conteneur
podman inspect web                                 # tout le détail en JSON
podman stop web && podman rm web                    # arrêter puis supprimer
podman rmi <image>                                  # supprimer une image
podman run -e VAR=valeur -v ./data:/data:Z <image> # variable + volume
podman build -t nom:tag .                           # construire
podman push nom:tag <registre>/nom:tag            # publier
```

LES DONNÉES D'UN CONTENEUR SONT ÉPHÉMÈRES

Tout ce qu'un conteneur écrit disparaît quand on le supprime. Pour garder des données, il faut un **volume**. C'est exactement le même principe dans Kubernetes (chapitre 8).

TU DOIS SAVOIR RÉPONDRE

- Quelle différence entre une image et un conteneur ? Entre un tag et un digest ?
- Pourquoi un conteneur s'arrête-t-il tout seul ?
- Où vont les données écrites dans un conteneur supprimé ?

CHAPITRE 03

Le YAML sans douleur

Le format de tous les fichiers Kubernetes. Trois règles suffisent.

■ Règle 1 : clé: valeur

```
name: web           # texte
replicas: 2         # nombre
enabled: true       # booléen (true / false)
port: "8080"        # texte, car entre guillemets
comment: ~          # null (vide)
```

Il faut **un espace après les deux-points**. `name:web` est une erreur.

■ Règle 2 : l'indentation crée la hiérarchie

```
metadata:           # "metadata" contient ce qui est indenté dessous
  name: web
  labels:            # "labels" est dans "metadata"
    app: web         # "app" est dans "labels"
```

- Uniquement des **espaces**, jamais de tabulations.
- Le nombre d'espaces est libre, mais **constant** au même niveau. Convention Kubernetes : **2 espaces**.
- Deux clés au même niveau d'indentation sont « sœurs ».

■ Règle 3 : les listes commencent par un tiret

```
ports:              # liste de nombres
- 8080
- 8443

containers:         # liste d'objets : chaque tiret ouvre un nouvel élément
- name: web          # élément 1
  image: httpd        # même élément (aligné sous "name")
- name: sidecar       # élément 2
  image: busybox
```

Écrire `- name: web` au même niveau que la clé parente (`containers:`) ou indenté de 2 espaces est valide. L'important est que les clés d'un même élément soient alignées.

■ Le reste, moins fréquent

```
# commentaire
text: |                # bloc multi-lignes, sauts de ligne conservés
  ligne 1
  ligne 2
---                  # sépare plusieurs documents (ressources) dans un fichier
ports: [8080, 8443]   # liste sur une ligne
labels: {app: web}    # dictionnaire sur une ligne
```

■ Anatomie d'un manifeste Kubernetes

Toutes les ressources ont la même structure de base :

```
apiVersion: apps/v1    # version de l'API qui gère ce type d'objet
kind: Deployment        # type d'objet
metadata:               # identité : nom, namespace, labels
  name: web
  labels:
    app: web
spec:                  # ce que TU veux (état désiré)
  replicas: 2
  ...
# status:              ce que Kubernetes constate (état réel) – rempli par lui, jamais par toi
```

NE JAMAIS TOUT ÉCRIRE DE MÉMOIRE

- Générer un squelette : `oc create deployment web --image=... --dry-run=client -o yaml > web.yaml`
- Voir un objet existant : `oc get deploy web -o yaml`
- Trouver un champ : `oc explain deployment.spec.template.spec.containers`
- Valider sans créer : `oc apply -f web.yaml --dry-run=server`

■ Les erreurs les plus fréquentes

Erreur	Symptôme
Tabulation au lieu d'espaces	<code>found character that cannot start any token</code>
Indentation décalée d'un espace	<code>mapping values are not allowed here</code> , ou champ ignoré / mal placé
Oubli de l'espace après <code>:</code>	La ligne devient un simple texte
Nombre là où Kubernetes attend un texte (ou l'inverse)	<code>cannot unmarshal number into ... string</code> – ex. valeur de variable d'env <code>value: 8080</code> au lieu de <code>"8080"</code>
Mauvaise casse	<code>containerport</code> $\neq$ <code>containerPort</code> : champ inconnu

TU DOIS SAVOIR RÉPONDRE

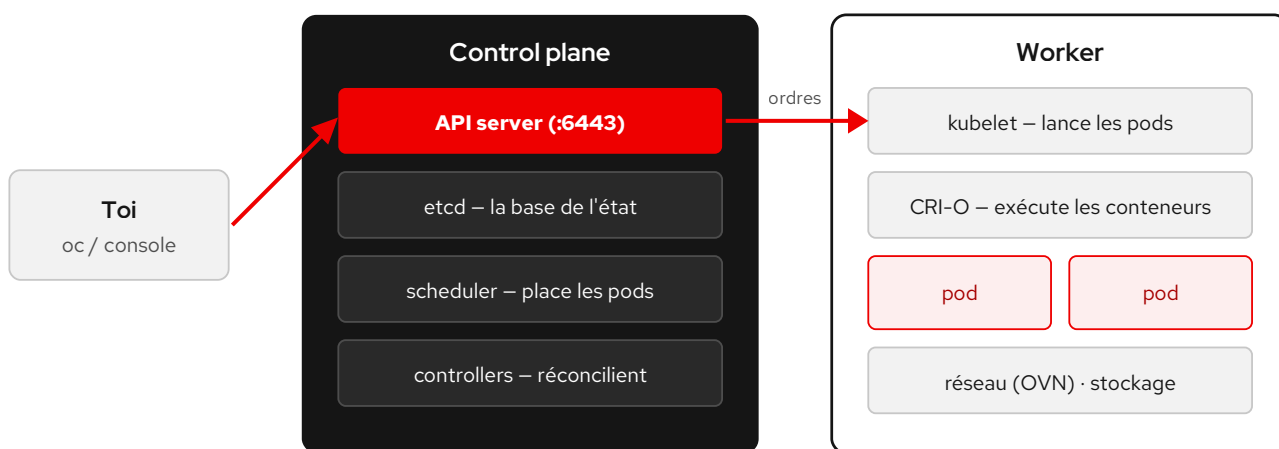
- Quelles sont les 3 règles du YAML ?
- Quelle différence entre `spec` et `status` ?
- Comment générer un YAML valide sans l'écrire entièrement ?

CHAPITRE 04

Kubernetes : architecture et fonctionnement

Qui fait quoi dans un cluster, et l'idée centrale : l'état désiré.

■ Les deux types de nœuds



Dans ton Single Node OpenShift, le control plane et le worker sont sur la même VM.

Composant	Rôle
API server	La seule porte d'entrée. <code>oc</code> , la console et les composants internes passent tous par lui.
etcd	La base de données qui contient tout l'état du cluster. La perdre, c'est perdre le cluster.
Scheduler	Choisit sur quel nœud lancer chaque nouveau pod (selon les ressources disponibles).
Controllers	Boucles qui comparent en permanence l'état désiré et l'état réel, et corrigent l'écart.
kubelet	Agent sur chaque nœud : reçoit la liste des pods à faire tourner et les lance via CRI-O.

■ L'idée centrale : l'état désiré

Avec Kubernetes, tu ne dis pas « lance ce conteneur ». Tu **déclares** ce que tu veux (« je veux 3 copies de cette application ») et Kubernetes s'occupe en permanence de rendre la réalité conforme.



La boucle de réconciliation tourne en permanence : c'est ce qui rend Kubernetes « auto-réparateur ».

CONSÉQUENCE PRATIQUE

Si tu supprimes un pod géré par un Deployment, il revient aussitôt : l'état désiré n'a pas changé. Pour arrêter une application, on modifie l'état désiré (`replicas: 0`) ou on supprime le Deployment.

Namespaces

Un **namespace** est un espace isolé dans le cluster : les noms n'y entrent pas en conflit avec ceux d'un autre namespace, et on peut y appliquer des droits et des quotas. Dans OpenShift, on parle de **projet** (un namespace enrichi). Chaque exercice se fait dans son propre projet, qu'on supprime à la fin.

Impératif ou déclaratif

Style	Exemple	Quand
Impératif	<code>oc create deployment web --image=...</code>	Rapide : tests, examen
Déclaratif	<code>oc apply -f web.yaml</code>	Reproductible, versionné dans Git : la production

TU DOIS SAVOIR RÉPONDRE

- À quoi servent l'API server, etcd, le scheduler et le kubelet ?
- Pourquoi un pod supprimé revient-il ?
- Qu'est-ce qu'un namespace, et comment s'appelle-t-il dans OpenShift ?

CHAPITRE 05

Pods, Deployments et labels

Faire tourner une application, puis la garder en vie.

■ Le Pod

Le **pod** est la plus petite unité de Kubernetes : un ou plusieurs conteneurs qui partagent la même adresse IP et peuvent partager des volumes. Dans 95 % des cas, un pod = un conteneur.

```
apiVersion: v1
kind: Pod
metadata:
  name: web
  labels:
    app: web
spec:
  containers:
  - name: web
    image: quay.io/redhattraining/hello-world-nginx
    ports:
    - containerPort: 8080
```

- Un pod reçoit une IP interne, qui **change** à chaque recreation.
- Un pod créé seul (« non géré ») n'est **pas recréé** s'il meurt ou si on le supprime.
- Cycle de vie : Pending → ContainerCreating → Running → (Succeeded / Failed).

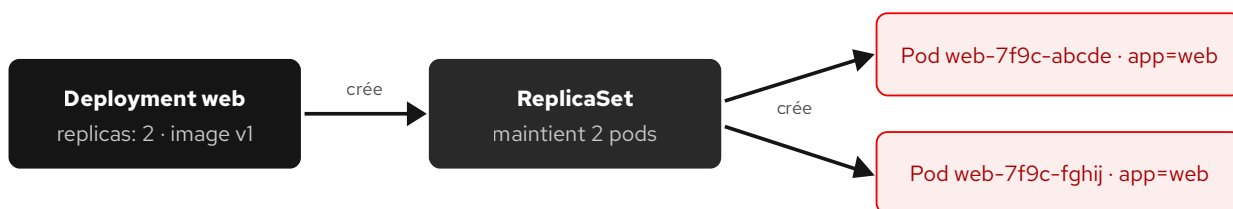
■ Le Deployment

En pratique, on ne crée presque jamais de pod à la main. On crée un **Deployment**, qui décrit le pod voulu et le nombre de copies. Le Deployment crée un **ReplicaSet**, qui maintient ce nombre de pods.

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 2                # nombre de copies voulu
  selector:
    matchLabels:
      app: web               # "les pods qui m'appartiennent ont app=web"
  template:                  # modèle de pod (même contenu que le Pod ci-dessus)
    metadata:
      labels:
        app: web            # DOIT correspondre au selector
    spec:
      containers:
        - name: web
          image: quay.io/redhattraining/hello-world-nginx
          ports:
            - containerPort: 8080

```



Le nom des pods reprend celui du ReplicaSet, suivi d'un suffixe aléatoire.

■ Les labels et les sélecteurs

Un **label** est une étiquette `clé=valeur` posée sur un objet. Un **sélecteur** est une requête sur ces étiquettes. C'est le mécanisme qui relie tout dans Kubernetes : un Deployment retrouve ses pods, un Service retrouve ses destinataires, uniquement grâce aux labels.

```

$ oc get pods --show-labels      # voir les labels
$ oc get pods -l app=web        # filtrer par label
$ oc label pod web-xyz env=test  # ajouter un label

```

PIÈGE CLASSIQUE

Si `selector.matchLabels` et `template.metadata.labels` ne correspondent pas, Kubernetes refuse le Deployment. Si le sélecteur d'un **Service** ne correspond à aucun pod, il n'y a pas d'erreur : l'application est simplement injoignable.

■ Les autres contrôleurs (à connaître de nom)

Objet	Usage
StatefulSet	Pods avec identité et stockage stables (bases de données en cluster).

Objet	Usage
DaemonSet	Un pod par nœud (agents de logs, de monitoring).
Job	Tâche qui s'exécute jusqu'au bout puis s'arrête.
CronJob	Job planifié (sauvegarde chaque nuit).

TU DOIS SAVOIR RÉPONDRE

- Pourquoi ne crée-t-on presque jamais de pod seul ?
- Quel est le lien entre Deployment, ReplicaSet et Pod ?
- À quoi servent les labels ? Que se passe-t-il si un sélecteur ne correspond à rien ?

CHAPITRE 06

Réseau : Services et Routes

Donner une adresse stable à une application, puis l'ouvrir vers l'extérieur.

■ Le problème

Les pods ont des IP qui changent à chaque recreation, et il y en a plusieurs. Une autre application ne peut donc pas les appeler directement.

■ Le Service : une adresse stable à l'intérieur du cluster

Un **Service** donne un nom DNS et une IP fixes devant un groupe de pods (trouvés par sélecteur), et répartit le trafic entre eux.

```

apiVersion: v1
kind: Service
metadata:
  name: web
spec:
  selector:
    app: web          # envoie le trafic aux pods app=web
  ports:
    - port: 8080      # port du Service
      targetPort: 8080 # port du conteneur

```

- Nom DNS dans le cluster : `web` (même namespace) ou `web.<namespace>.svc.cluster.local` (partout).
- Les **endpoints** sont la liste des IP de pods actuellement derrière le Service : `oc get endpoints web`. S'ils sont vides, rien ne peut répondre.

Type	Accessible depuis
ClusterIP (défaut)	L'intérieur du cluster uniquement. Le cas normal.
NodePort	Un port (30000–32767) ouvert sur chaque nœud.
LoadBalancer	Une IP externe fournie par le cloud ou MetalLB.

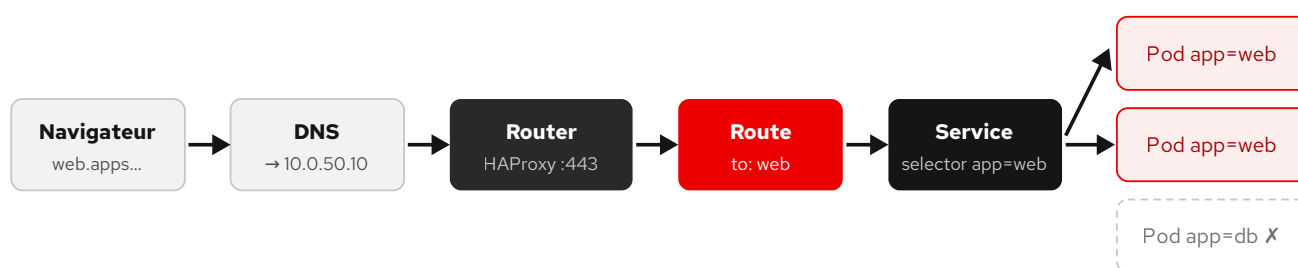
■ La Route : ouvrir vers l'extérieur (OpenShift)

Une **Route** associe un nom d'hôte public à un Service. Le **router** d'OpenShift (HAProxy, ports 80 et 443) reçoit les requêtes et les transmet. C'est pour cela que tu as créé le DNS wildcard `*.apps.ocp.lab.example.com`.

```

apiVersion: route.openshift.io/v1
kind: Route
metadata:
  name: web
spec:
  host: web.apps.ocp.lab.example.com      # facultatif : généré sinon (web-<projet>.apps...)
  to:
    kind: Service
    name: web
  port:
    targetPort: 8080
  tls:                                     # facultatif : HTTPS
    termination: edge
    insecureEdgeTerminationPolicy: Redirect

```



Le Service ne sélectionne que les pods dont les labels correspondent (le pod app=db est ignoré).

■ Les trois types de TLS d'une Route

Type	Où s'arrête le chiffrement
edge	Au router. Le router parle ensuite en HTTP au pod. Le plus courant.
passthrough	Au pod : le router ne déchiffre rien, l'application gère son certificat.
reencrypt	Au router, qui rechiffre vers le pod avec un autre certificat.

ET L'INGRESS ?

L'**Ingress** est l'équivalent standard de la Route dans Kubernetes. OpenShift le supporte aussi (il le convertit en Route), mais on utilise surtout des Routes.

TU DOIS SAVOIR RÉPONDRE

- Pourquoi a-t-on besoin d'un Service devant des pods ?
- Que sont les endpoints ? Que signifie une liste vide ?
- Décris le trajet d'une requête du navigateur jusqu'au pod.

CHAPITRE 07

Configuration : ConfigMaps et Secrets

Séparer la configuration de l'image, pour utiliser la même image partout.

Une bonne image ne contient **aucune** configuration propre à un environnement (adresse de base de données, mot de passe, mode debug). On la fournit au lancement, depuis deux types d'objets :

Objet	Contenu	Remarque
ConfigMap	Configuration non sensible : variables, fichiers de config.	Lisible par tous ceux qui ont accès au projet.
Secret	Données sensibles : mots de passe, tokens, certificats.	Encodé en base64 , ce qui n'est pas du chiffrement ; l'accès est protégé par les droits.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: web-config
data:
  APP_MODE: "production"
  app.conf: |
    log_level=info
    max_users=50
---
apiVersion: v1
kind: Secret
metadata:
  name: db-creds
type: Opaque
stringData:
  DB_USER: app
  DB_PASSWORD: S3cret
```

en clair ici ; stocké en base64 par Kubernetes

■ Deux façons de les utiliser

```
spec:
  containers:
  - name: web
    image: ...
    envFrom:
      # 1. toutes les clés en variables d'environnement
      - configMapRef:
          name: web-config
      - secretRef:
          name: db-creds
    volumeMounts:
      # 2. en fichiers dans un dossier
      - name: config
        mountPath: /etc/app
      # → /etc/app/app.conf, /etc/app/APP_MODE
  volumes:
  - name: config
    configMap:
      name: web-config
```

En commande : `oc set env deploy/web --from=configmap/web-config` et `oc set volumes deploy/web --add --type=configmap --configmap-name=web-config --mount-path=/etc/app`.

MISE À JOUR DE LA CONFIGURATION

Des **variables d'environnement** ne sont lues qu'au démarrage du conteneur : après une modification, il faut redémarrer les pods (`oc rollout restart deploy/web`). Les **fichiers montés** sont mis à jour automatiquement après quelques secondes, mais l'application doit les relire.

TU DOIS SAVOIR RÉPONDRE

- Pourquoi sortir la configuration de l'image ?
- Quelle différence entre ConfigMap et Secret ? Un Secret est-il chiffré ?
- Quelles sont les deux façons de fournir une ConfigMap à un conteneur ?

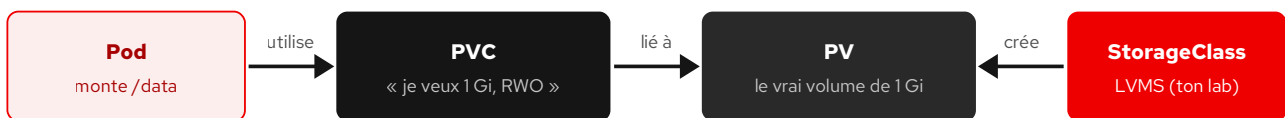
CHAPITRE 08

Stockage persistant

Garder des données quand les pods disparaissent.

Comme avec Podman, tout ce qu'un conteneur écrit disparaît avec lui. Une base de données ou des fichiers envoyés par des utilisateurs doivent vivre sur un **volume persistant**, indépendant du pod.

■ Les trois objets



Provisionnement dynamique : il suffit de créer le PVC, la StorageClass crée le PV automatiquement.

Objet	Rôle	Qui le crée
PersistentVolumeClaim (PVC)	La demande : taille, mode d'accès, type de stockage.	Toi (l'application)
PersistentVolume (PV)	Le volume réel, lié à un seul PVC.	La StorageClass (ou un admin)
StorageClass (SC)	Le « fournisseur » : sait créer des PV d'un certain type.	L'admin du cluster

```

apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: data
spec:
  accessModes: [ReadWriteOnce]
  resources:
    requests:
      storage: 1Gi
  # storageClassName: lvms-vg1 # facultatif si une StorageClass est "default"
---
# dans le Deployment (spec.template.spec) :
  containers:
    - name: web
      volumeMounts:
        - name: data
          mountPath: /data
      volumes:
        - name: data
          persistentVolumeClaim:
            claimName: data

```

■ Modes d'accès

Mode	Signification
ReadWriteOnce (RWO)	Lecture/écriture par un seul nœud à la fois. Le plus courant (bases de données). Seul mode proposé par LVMS.
ReadWriteMany (RWX)	Lecture/écriture par plusieurs nœuds (NFS, CephFS). Utile pour des fichiers partagés.
ReadOnlyMany (ROX)	Lecture seule par plusieurs nœuds.

ÉTATS D'UN PVC

Pending : en attente d'un volume (mauvaise StorageClass, mode non supporté, ou volume créé seulement quand un pod l'utilise). **Bound** : lié à un PV, utilisable.

TU DOIS SAVOIR RÉPONDRE

- Quel est le rôle du PVC, du PV et de la StorageClass ?
- Que signifie RWO ? Pourquoi une base de données l'utilise-t-elle ?
- Que deviennent les données d'un PVC quand on supprime le pod ?

CHAPITRE 09

Fiabilité et mises à jour

Une application qui se répare, tient la charge et se met à jour sans coupure.

■ Les sondes (probes)

Un conteneur « Running » n'est pas forcément en état de répondre. Les sondes permettent à Kubernetes de le vérifier.

Sonde	Question posée	Si elle échoue
readiness	Es-tu prêt à recevoir du trafic ?	Le pod est retiré des endpoints du Service (plus de trafic), sans redémarrage.
liveness	Es-tu encore vivant ?	Le conteneur est redémarré .
startup	As-tu fini de démarrer ?	Les autres sondes attendent son succès (applis lentes à démarrer).

Méthodes de test : **httpGet** (réponse HTTP 200–399), **tcpSocket** (le port accepte une connexion), **exec** (une commande renvoie 0).

■ Ressources : requests et limits

Réglage	Effet
requests	Ce qui est réservé pour le conteneur. Le scheduler ne place un pod que sur un nœud qui a ces ressources libres.
limits	Le plafond . CPU dépassé : le conteneur est ralenti. Mémoire dépassée : il est tué (<code>OOMKilled</code>).

Unités : CPU en **millicœurs** (`100m`) = 0,1 cœur ; `1` = 1 cœur). Mémoire en **Mi/Gi** (`128Mi` , `1Gi`).

```

containers:
- name: web
  image: quay.io/redhattraining/hello-world-nginx
  resources:
    requests:
      cpu: 100m
      memory: 128Mi
    limits:
      cpu: 500m
      memory: 256Mi
  readinessProbe:
    httpGet:
      path: /
      port: 8080
    initialDelaySeconds: 5
    periodSeconds: 10
  livenessProbe:
    tcpSocket:
      port: 8080
    periodSeconds: 10

```

■ L'autoscaling (HPA)

Le **HorizontalPodAutoscaler** ajoute ou retire des pods selon la charge. Le pourcentage CPU est calculé par rapport aux **requests** : sans requests, il ne fonctionne pas.

```

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: web
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: web
  minReplicas: 1
  maxReplicas: 5
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70

```

Équivalent en une commande : `oc autoscale deploy/web --min=1 --max=5 --cpu-percent=70`.

■ Les mises à jour (rollouts)

Quand on change le modèle de pod d'un Deployment (nouvelle image, nouvelle variable), Kubernetes crée un **nouveau ReplicaSet** et bascule progressivement.

Stratégie	Comportement
RollingUpdate (défaut)	Remplace les pods un par un : aucune coupure. Si les nouveaux pods ne deviennent jamais « Ready », les anciens continuent de servir.
Recreate	Arrête tous les anciens pods, puis lance les nouveaux : coupure courte. Utile avec un volume RWO.

```
$ oc set image deploy/web web=<nouvelle-image> # déclenche un rollout
$ oc rollout status deploy/web                 # suivre
$ oc rollout history deploy/web                 # révisions
$ oc rollout undo deploy/web                    # revenir à la précédente
```

TU DOIS SAVOIR RÉPONDRE

- Quelle différence entre readiness et liveness ?
- Que se passe-t-il si un conteneur dépasse sa limite mémoire ? sa limite CPU ?
- Pourquoi l'HPA a-t-il besoin des requests ?
- Comment revenir à la version précédente d'une application ?

CHAPITRE 10

Ce qu'OpenShift ajoute

Les notions propres à OpenShift, au-dessus de Kubernetes.

Notion	Ce que c'est	À retenir
Project	Un namespace avec des métadonnées et des droits par défaut.	<code>oc new-project demo</code> crée et bascule dedans.
Route	Exposition HTTP(S) native (chapitre 6).	<code>oc expose svc/web</code>
SCC	Security Context Constraints : ce qu'un pod a le droit de faire.	Par défaut, pas de root et un UID aléatoire.
Template	Ensemble paramétrable de ressources (ex. une base MySQL complète).	<code>oc new-app --template=... -p PARAM=val</code>
ImageStream	Pointeur vers des images, avec des tags ; peut déclencher un redéploiement quand l'image change.	<code>oc get is -n openshift</code>
BuildConfig / S2I	Construction d'images dans le cluster, depuis un Git (Source-to-Image) ou un Containerfile.	Hors examen niveau 1.
Operators	Programmes qui installent et maintiennent un composant (le cluster lui-même, ou LVMS, une base de données...).	Catalogue : OperatorHub dans la console.
Console web	Vues Administrator et Developer (Topology).	Idéale pour visualiser les liens entre objets.
OAuth intégré	Utilisateurs, groupes, fournisseurs d'identité (HTPasswd, LDAP...).	<code>oc login -u student</code>
oc	La CLI : toutes les commandes <code>kubectl</code> + des commandes OpenShift.	Chapitre 11.

■ Le piège n°1 : les images qui veulent être root

Beaucoup d'images de Docker Hub (`nginx`, `httpd`, `mysql`) démarrent en root ou écoutent sur le port 80. Sur OpenShift, elles échouent (`CrashLoopBackOff`, « Permission denied »). La solution au niveau 1 est d'utiliser des images compatibles :

- les images Red Hat **UBI** (`registry.access.redhat.com/ubi9/...`) ;
- les images de `registry.redhat.io` (catalogue sur `catalog.redhat.com`) ;
- les images prévues « non-root », qui écoutent sur 8080.

■ **oc new-app** : le raccourci OpenShift

`oc new-app --image=<image>` crée en une commande un **Deployment** et un **Service** (et souvent une `ImageStream`), avec les bons labels. Il suffit ensuite de `oc expose svc/<nom>` pour avoir une Route.

TU DOIS SAVOIR RÉPONDRE

- Cite 4 notions propres à OpenShift.
- Pourquoi l'image `nginx` de Docker Hub échoue-t-elle sur OpenShift ?
- Que crée `oc new-app` ?

CHAPITRE 11

Premiers pas avec oc et vim

Les 20 commandes du quotidien et de quoi éditer un YAML proprement.

■ La forme d'une commande oc

```
oc <verbe> <type>/<nom> [options]
oc get pods -n demo
oc describe deployment/web
oc delete pod/web-7f9c-abcde
```

■ Les commandes indispensables

Commande	Rôle
<code>oc login -u <user> <url></code>	Se connecter
<code>oc whoami</code> · <code>oc project</code>	Qui suis-je ? Dans quel projet ?
<code>oc new-project demo</code>	Créer un projet et y entrer
<code>oc get pods</code> · <code>oc get all</code>	Lister
<code>oc get pod <nom> -o yaml</code>	Voir un objet complet
<code>oc describe pod <nom></code>	Détail lisible + Events (le premier réflexe de dépannage)
<code>oc logs <pod></code> · <code>-f</code> · <code>--previous</code>	Journaux (en continu / du conteneur précédent)
<code>oc exec <pod> -- <cmd></code> · <code>oc rsh <pod></code>	Exécuter une commande / ouvrir un shell
<code>oc apply -f fichier.yaml</code>	Créer ou mettre à jour depuis un fichier
<code>oc delete -f fichier.yaml</code>	Supprimer ce que décrit le fichier
<code>oc create deployment ... --dry-run=client -o yaml</code>	Générer un YAML sans rien créer
<code>oc explain <type>.<champ></code>	Documentation d'un champ
<code>oc edit deploy/web</code>	Modifier un objet en direct dans l'éditeur
<code>oc get events --sort-by=.lastTimestamp</code>	Ce qui s'est passé récemment dans le projet
<code>oc delete project demo</code>	Tout supprimer d'un coup

■ Méthode de dépannage en 4 temps

1. `oc get pods` : quel est l'état ? (`Pending` , `CrashLoopBackOff` , `ImagePullBackOff` , `0/1 Ready`)
2. `oc describe pod <nom>` : lire la section **Events** en bas.
3. `oc logs <nom>` (et `--previous` si le conteneur redémarre en boucle).
4. Si le pod tourne mais l'appli est injoignable : `oc get svc, endpoints, route` .

■ vim pour le YAML

Crée `~/vimrc` une fois pour toutes :

```
set expandtab tabstop=2 shiftwidth=2 autoindent
set number cursorcolumn
```

Touche(s)	Action
<code>i</code> / <code>Échap</code>	Passer en mode insertion / revenir en mode commande
<code>:w</code> · <code>:q</code> · <code>:wq</code> · <code>:q!</code>	Enregistrer · quitter · les deux · quitter sans enregistrer
<code>o</code> / <code>O</code>	Nouvelle ligne dessous / dessus, en insertion
<code>dd</code> · <code>yy</code> · <code>p</code>	Couper · copier · coller une ligne
<code>u</code> · <code>Ctrl+r</code>	Annuler · rétablir
<code>>></code> · <code><<</code>	Indenter · désindenter la ligne (de 2 espaces avec le <code>.vimrc</code>)
<code>V</code> puis <code>></code>	Sélectionner des lignes puis les indenter en bloc
<code>/mot</code> · <code>n</code>	Chercher · occurrence suivante
<code>:set paste</code>	Avant de coller du texte, pour ne pas casser l'indentation
<code>gg</code> · <code>G</code>	Début · fin du fichier

CHAPITRE 12

Rappels réseau utiles

Ce qu'il faut avoir en tête pour diagnostiquer une application injoignable.

■ Les codes HTTP à reconnaître

Code	Signification	Cause fréquente dans OpenShift
200	OK	Tout fonctionne.
301 / 302	Redirection	Route edge avec redirection HTTP → HTTPS : normal.
403	Interdit	L'application refuse (droits sur les fichiers, configuration).
404	Introuvable	Mauvais chemin dans l'URL, ou nom d'hôte ne correspondant à aucune Route.
502	Mauvaise passerelle	Le router atteint le pod mais l'application répond mal (mauvais port, mauvais protocole).
503	Service indisponible	« Application is not available » : le Service n'a aucun endpoint (sélecteur faux, pods pas Ready).

■ Les outils de test

```
$ nslookup web.apps.ocp.lab.example.com      # le nom résout-il vers la bonne IP ?
$ curl -I http://web.apps.ocp.lab.example.com # code HTTP seulement
$ curl -kv https://web.apps.ocp.lab.example.com # détail TLS (-k : ignore le certificat)
$ oc run -it --rm t --image=registry.access.redhat.com/ubi9/ubi -- curl -s http://web:8080
# tester le Service depuis l'intérieur du cluster
```

■ Méthode : de l'extérieur vers l'intérieur

1. **DNS** : le nom résout-il vers le router (10.0.50.10) ?
2. **Route** : existe-t-elle avec ce nom d'hôte ? (`oc get route`)
3. **Service** : a-t-il des endpoints ? (`oc get endpoints`)
4. **Pods** : sont-ils Ready ? écoutent-ils sur le bon port ?

■ Ports à connaître dans ton lab

Port	Usage
6443	API du cluster (<code>oc login</code>)

Port	Usage
443 / 80	Router : console et applications (<code>*.apps</code>)
8080 / 8443	Ports d'écoute habituels des conteneurs non-root
22	SSH de secours vers le nœud (<code>core@</code>)

CHAPITRE 13

Exercices niveau zéro (13)

Du conteneur local au déploiement complet. Solutions en annexe.

ORGANISATION

Z01 et Z02 se font sur ton PC Fedora avec Podman. À partir de Z04, crée un projet dédié : `oc new-project bases`, et écris les fichiers YAML **à la main** dans `vim`, dans un dossier `~/bases/`. Difficulté : ★ facile, ★★ moyen, ★★★ synthèse.

Z01 Premier conteneur avec Podman

★ · ch. 2

1. Lance en arrière-plan un conteneur `web` depuis `registry.access.redhat.com/ubi9/httpd-24`, en publiant le port 8080.
2. Teste-le avec `curl`, puis lis ses logs.
3. Ouvre un shell dedans et trouve sous quel utilisateur tourne le processus (`id`).
4. Arrête-le et supprime-le.

Z02 Construire ta propre image

★ · ch. 2

1. Crée un fichier `index.html` contenant « Bonjour depuis mon image ».
2. Écris un `Containerfile` basé sur `ubi9/httpd-24` qui copie ce fichier dans `/var/www/html/`.
3. Construis l'image `monsite:1.0`, lance-la et vérifie avec `curl`.

Z03 Corriger un YAML cassé

★ · ch. 3

Ce fichier contient **3 erreurs**. Trouve-les sans l'exécuter, puis vérifie avec `oc apply --dry-run=server -f`.

```
apiVersion: v1
kind: Pod
metadata:
  name: broken
  labels:
    app: test
spec:
  containers:
  - name: web
    image: quay.io/redhattraining/hello-world-nginx
    ports:
    - containerPort: "8080"
```

Z04 Ton premier Pod en YAML

★ · ch. 5

1. Écris à la main `pod.yaml` : un pod `hello`, label `app=hello`, image `quay.io/redhattraining/hello-world-nginx`, port 8080.
2. Applique-le, puis affiche son état, son IP et ses logs.
3. Supprime-le. Revient-il ? Pourquoi ?

Z05 Ton premier Deployment

★★ · ch. 5

1. Écris `deploy.yaml` : Deployment `hello`, 2 réplicas, même image.
2. Applique-le et liste le Deployment, le ReplicaSet et les pods.
3. Supprime un pod : que se passe-t-il ?
4. Passe à 4 réplicas **en modifiant le fichier** puis en le réappliquant.

Z06 Un Service et son nom DNS

★★ · ch. 6

1. Écris `svc.yaml` : Service `hello` qui sélectionne `app=hello`, port 8080.
2. Vérifie ses endpoints : combien y en a-t-il ?
3. Depuis un pod temporaire, appelle `http://hello:8080`.

Z07 Une Route vers ton navigateur

★★ · ch. 6

1. Écris `route.yaml` : Route `hello`, hôte `hello.apps.ocp.lab.example.com`, vers le Service `hello`, en HTTPS edge avec redirection.
2. Ouvre `https://hello.apps.ocp.lab.example.com` dans ton navigateur.
3. Regarde le résultat dans la vue **Topology** de la console.

Z08 Casser et réparer les labels

★★ · ch. 5 et 6

1. Dans `svc.yaml`, remplace le sélecteur par `app: he'lo` (faute de frappe) et réapplique.
2. Observe : la page web, les endpoints, les erreurs affichées (y en a-t-il ?).
3. Répare.

Z09 ConfigMap et Secret

★★ · ch. 7

1. Écris `config.yaml` contenant une ConfigMap `hello-config` (`COLOR: blue`) et un Secret `hello-secret` (`API_KEY: abc123`), séparés par `---`.
2. Injecte les deux en variables dans le Deployment `hello` (via `envFrom` dans `deploy.yaml`).
3. Vérifie les variables dans un pod. Change `COLOR` : que faut-il faire pour que les pods la voient ?

Z10 Un volume qui survit

★★ · ch. 8

1. Liste les StorageClasses et repère celle par défaut.
2. Écris `pvc.yaml` : PVC `notes` de 1 Gi en RWO.
3. Écris un Deployment `notes` (1 réplica, image `registry.access.redhat.com/ubi9/ubi`, commande `sleep infinity`) qui monte ce PVC dans `/data`.
4. Écris un fichier dans `/data`, supprime le pod, vérifie que le fichier existe toujours.

Z11 Sondes et ressources

★★ · ch. 9

1. Ajoute au Deployment `hello` : requests 50m / 64Mi, limits 200m / 128Mi, une readiness HTTP sur `/` port 8080 et une liveness TCP sur 8080.
2. Change la readiness pour le chemin `/nexistepas` : que voit-on dans `oc get pods` et dans les endpoints ?
3. Remets le bon chemin.

Z12 Mettre à jour et revenir en arrière

★★ · ch. 9

1. Change l'image de `hello` pour une image qui n'existe pas (`.../hello-world-nginx:v999`).
2. Observe : les anciens pods servent-ils encore la page ? Pourquoi ?
3. Reviens en arrière avec `oc rollout undo` et consulte l'historique.

Z13 L'application complète en un fichier

★★★ · synthèse

Dans un nouveau projet `bases-final`, écris un seul fichier `app.yaml` (documents séparés par `---`) contenant : une ConfigMap, un Secret, un PVC de 1 Gi, un Deployment de 2 réplicas utilisant les trois (variables + volume `/data`, requests/limits, readiness et liveness), un Service et une Route edge sur `final.apps.ocp.lab.example.com`. Déploie tout avec **une seule** commande, vérifie dans le navigateur, puis supprime tout avec **une seule** commande.

Avec 2 réplicas et un volume RWO sur un seul nœud, cela fonctionne dans ton SNO ; sur un cluster à plusieurs nœuds, ce ne serait pas garanti.

CHAPITRE 14

Glossaire

Les mots à connaître, par ordre alphabétique.

Terme	Définition
API server	Point d'entrée unique du cluster (port 6443).
ConfigMap	Objet contenant de la configuration non sensible.
Conteneur	Processus isolé lancé depuis une image.
Control plane	Ensemble des composants qui pilotent le cluster.
CRI-O	Moteur qui exécute les conteneurs sur les nœuds OpenShift.
Deployment	Objet qui déclare une application (image, réplicas) et gère ses mises à jour.
Digest	Empreinte sha256 immuable d'une image.
Endpoints	Liste des IP de pods derrière un Service.
etcd	Base de données clé/valeur de l'état du cluster.
HPA	HorizontalPodAutoscaler : ajuste le nombre de réplicas selon la charge.
Image	Modèle en lecture seule d'un conteneur.
ImageStream	Objet OpenShift qui référence des images par tags.
kubelet	Agent de chaque nœud qui lance les pods.
Label / sélecteur	Étiquette clé=valeur / requête sur ces étiquettes.
Limits / requests	Plafond / réservation de CPU et mémoire.
Manifeste	Fichier YAML décrivant une ou plusieurs ressources.
Namespace / Project	Espace isolé dans le cluster (Project = version OpenShift).
Nœud (node)	Machine (physique ou VM) du cluster.
OOMKilled	Conteneur tué pour dépassement de sa limite mémoire.
Operator	Programme qui installe et maintient un composant.
Pod	Plus petite unité : un ou plusieurs conteneurs partageant une IP.
Probe	Test de santé (readiness, liveness, startup).
PV / PVC	Volume persistant / demande de volume.
Registre	Serveur qui stocke les images.

Terme	Définition
ReplicaSet	Maintient un nombre donné de pods identiques.
Rollout / rollback	Déploiement d'une nouvelle version / retour à la précédente.
Route	Exposition externe HTTP(S) d'un Service (OpenShift).
Router	HAProxy d'OpenShift qui reçoit le trafic externe.
SCC	Security Context Constraints : règles de sécurité des pods (OpenShift).
Scheduler	Choisit le nœud de chaque pod.
Secret	Objet contenant des données sensibles (encodées en base64).
Service	Adresse et nom DNS stables devant un groupe de pods.
SNO	Single Node OpenShift : cluster sur un seul nœud (ton lab).
StorageClass	Fournisseur de volumes à la demande.
Tag	Étiquette mobile d'une image (<code>latest</code> , <code>1.0</code>).
UBI	Universal Base Image : images de base Red Hat, libres et compatibles OpenShift.

ANNEXE

Solutions des exercices

À ne lire qu'après avoir essayé (au moins 10 minutes par exercice).

Z01 – Premier conteneur avec Podman

```
$ podman run -d --name web -p 8080:8080 registry.access.redhat.com/ubi9/httpd-24
$ curl -s localhost:8080 | head -5; podman logs web
$ podman exec -it web bash
bash-5.1$ id                # uid=1001(default) : l'image ne tourne pas en root
bash-5.1$ exit
$ podman stop web && podman rm web
```

Z02 – Construire ta propre image

```
$ echo 'Bonjour depuis mon image' > index.html
$ cat > Containerfile <<'EOF'
FROM registry.access.redhat.com/ubi9/httpd-24
COPY index.html /var/www/html/
EOF
$ podman build -t monsite:1.0 .
$ podman run -d --name monsite -p 8080:8080 monsite:1.0 && curl localhost:8080
```

Z03 – Corriger un YAML cassé

1. `labels:` est indenté d'**un** espace au lieu de deux : il doit être aligné sous `name`.
2. `ports:` est trop indenté : il doit être aligné sous `image` (même élément de liste).
3. `containerPort: "8080"` est un texte ; Kubernetes attend un **nombre** : `containerPort: 8080`.

```
metadata:
  name: broken
  labels:
    app: test
spec:
  containers:
    - name: web
      image: quay.io/redhattraining/hello-world-nginx
      ports:
        - containerPort: 8080
```

Z04 – Ton premier Pod en YAML

```
$ oc new-project bases
$ vim pod.yaml          # contenu : exemple du chapitre 5, avec name/label "hello"
$ oc apply -f pod.yaml
$ oc get pod hello -o wide; oc logs hello
$ oc delete pod hello; oc get pods    # il ne revient pas : aucun contrôleur ne le gère
```

Z05 – Ton premier Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hello
spec:
  replicas: 2
  selector:
    matchLabels:
      app: hello
  template:
    metadata:
      labels:
        app: hello
    spec:
      containers:
        - name: hello
          image: quay.io/redhattraining/hello-world-nginx
          ports:
            - containerPort: 8080
$ oc apply -f deploy.yaml
$ oc get deploy,rs,pods
$ oc delete pod <un-pod-hello>; oc get pods -w    # un nouveau pod est créé aussitôt
# modifier replicas: 4 dans le fichier, puis :
$ oc apply -f deploy.yaml && oc get pods
```

Z06 – Un Service et son nom DNS

```
apiVersion: v1
kind: Service
metadata:
  name: hello
spec:
  selector:
    app: hello
  ports:
    - port: 8080
      targetPort: 8080
$ oc apply -f svc.yaml
$ oc get endpoints hello          # autant d'adresses que de pods (4)
$ oc run -it --rm t --image=registry.access.redhat.com/ubi9/ubi -- curl -s http://hello:8080
```

Z07 – Une Route vers ton navigateur

```

apiVersion: route.openshift.io/v1
kind: Route
metadata:
  name: hello
spec:
  host: hello.apps.ocp.lab.example.com
  to:
    kind: Service
    name: hello
  port:
    targetPort: 8080
  tls:
    termination: edge
    insecureEdgeTerminationPolicy: Redirect
$ oc apply -f route.yaml
$ curl -kI https://hello.apps.ocp.lab.example.com # 200
# Console : vue Developer > Topology, projet "bases"

```

Z08 – Casser et réparer les labels

Après la faute de frappe : **aucune erreur** n'est affichée à l'application du fichier, mais `oc get endpoints hello` est vide et la page affiche « Application is not available » (503). C'est pour cela que ce bug est piégeux. Réparation : remettre `app: hello` et réappliquer `svc.yaml`.

Z09 – ConfigMap et Secret

```

apiVersion: v1
kind: ConfigMap
metadata:
  name: hello-config
data:
  COLOR: blue
---
apiVersion: v1
kind: Secret
metadata:
  name: hello-secret
stringData:
  API_KEY: abc123
# dans deploy.yaml, sous le conteneur (même niveau que image:) :
  envFrom:
    - configMapRef:
        name: hello-config
    - secretRef:
        name: hello-secret
$ oc apply -f config.yaml -f deploy.yaml
$ oc exec deploy/hello -- env | grep -E 'COLOR|API_KEY'
# après modification de COLOR : oc apply -f config.yaml puis oc rollout restart deploy/hello

```

Z10 – Un volume qui survit

```
$ oc get sc                                     # celle marquée (default)
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: notes
spec:
  accessModes: [ReadWriteOnce]
  resources:
    requests:
      storage: 1Gi
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: notes
spec:
  replicas: 1
  selector:
    matchLabels:
      app: notes
  template:
    metadata:
      labels:
        app: notes
    spec:
      containers:
        - name: notes
          image: registry.access.redhat.com/ubi9/ubi
          command: ["sleep", "infinity"]
          volumeMounts:
            - name: data
              mountPath: /data
      volumes:
        - name: data
          persistentVolumeClaim:
            claimName: notes
$ oc apply -f pvc.yaml; oc get pvc notes        # Bound
$ oc exec deploy/notes -- sh -c 'echo test > /data/f.txt'
$ oc delete pod -l app=notes; oc get pods -w
$ oc exec deploy/notes -- cat /data/f.txt       # "test"
```

Z11 – Sondes et ressources

```
# dans deploy.yaml, sous le conteneur :
resources:
  requests: {cpu: 50m, memory: 64Mi}
  limits: {cpu: 200m, memory: 128Mi}
readinessProbe:
  httpGet: {path: /, port: 8080}
  initialDelaySeconds: 5
livenessProbe:
  tcpSocket: {port: 8080}
  periodSeconds: 10

$ oc apply -f deploy.yaml

# avec path: /nexistepas → les nouveaux pods restent 0/1 Ready, hors des endpoints ;
# le rolling update s'arrête et les anciens pods continuent de servir. Remettre "/" et
réappliquer.
```

Z12 – Mettre à jour et revenir en arrière

```
$ oc set image deploy/hello hello=quay.io/redhattraining/hello-world-nginx:v999
$ oc get pods          # nouveau pod en ImagePullBackOff, anciens toujours Running
$ curl -kI https://hello.apps.ocp.lab.example.com # 200 : RollingUpdate ne retire les
anciens pods

# que lorsque les nouveaux sont Ready

$ oc rollout undo deploy/hello
$ oc rollout history deploy/hello
```

Z13 – L'application complète en un fichier

Assemble les solutions Z05, Z06, Z07, Z09, Z10 et Z11 dans un seul fichier, séparées par `---`, en adaptant les noms et l'hôte `final.apps.ocp.lab.example.com`. Le Deployment doit avoir `envFrom` (ConfigMap + Secret), `volumeMounts` / `volumes` (PVC), `resources` et les deux sondes.

```
$ oc new-project bases-final
$ oc apply -f app.yaml          # tout est créé d'un coup
$ oc get all,cm,secret,pvc,route
$ curl -kI https://final.apps.ocp.lab.example.com
$ oc delete -f app.yaml        # tout est supprimé d'un coup
```

PROCHAINE ÉTAPE

Quand Z13 fonctionne sans aide, tu as les bases. Passe au guide « OpenShift niveau 1 (DO180 / EX180) » : ses exercices E01 à E30 te paraîtront beaucoup plus simples.